

Frontend Tag 15

Spread- und Rest-Operator in JavaScript

Lernziel: Du kannst Werte mit Spread verteilen und mit Rest einsammeln. Beide verwenden dieselben drei Punkte (...), aber in unterschiedlichem Kontext.

SPREAD	REST
verteilt / kopiert vorhandene Werte	sammelt übrige Werte
Arrays, Objekte, Funktionsaufrufe	Destructuring, Funktionsparameter

1. Spread bei Arrays

Der Spread-Operator entpackt die einzelnen Elemente eines Arrays an einer neuen Stelle.

Arrays kopieren

So entsteht ein neues Array. Das Original bleibt unverändert.

```
const skills = ["HTML", "CSS", "JavaScript"];
const kopie = [...skills];

console.log(kopie); // ["HTML", "CSS", "JavaScript"]
```

Merke: Bei verschachtelten Arrays oder Objekten ist dies nur eine flache Kopie (shallow copy).

Arrays zusammenführen

Mehrere Arrays lassen sich ohne concat() zu einem neuen Array kombinieren.

```
const frontend = ["HTML", "CSS"];
const tools = ["Git", "VS Code"];

const stack = [...frontend, ...tools];
// ["HTML", "CSS", "Git", "VS Code"]
```

Zusätzliche Werte einfügen

Neue Werte können vor, zwischen oder nach den verteilten Elementen stehen.

```
const basics = ["HTML", "CSS"];

const erweitert = ["Git", ...basics, "JavaScript"];
// ["Git", "HTML", "CSS", "JavaScript"]
```

Merke: Die Position von ...basics bestimmt die Reihenfolge im neuen Array.

2. Spread bei Objekten

Bei Objekten verteilt Spread die eigenen, aufzählbaren Properties in ein neues Objekt.

Objekte kopieren

Die vorhandenen Schlüssel-Wert-Paare werden übernommen.

```
const developer = {  
  name: "Marcel",  
  role: "Frontend-Entwickler"  
};  
  
const kopie = { ...developer };
```

Properties ergänzen

Zusätzliche Properties werden direkt im neuen Objekt angegeben.

```
const erweitert = {  
  ...developer,  
  erfahrung: "Einsteiger",  
  aktiv: true  
};
```

Properties überschreiben und Reihenfolge

Kommt derselbe Schlüssel mehrfach vor, gewinnt der zuletzt angegebene Wert.

```
const original = { name: "Marcel", role: "Lernender" };  
  
const a = { ...original, role: "Frontend-Entwickler" };  
// role: "Frontend-Entwickler"  
  
const b = { role: "Frontend-Entwickler", ...original };  
// role: "Lernender"
```

Merke: Lies ein Objektliteral von links nach rechts: spätere Properties überschreiben frühere.

3. Rest: übrige Werte sammeln

Rest steht beim Destructuring oder in Parametern. Es sammelt alles ein, was noch nicht einzeln zugewiesen wurde.

Rest bei Arrays

Die ersten Elemente können einzeln benannt werden; der Rest wird als neues Array gesammelt.

```
const skills = ["HTML", "CSS", "JavaScript", "React"];

const [erste, zweite, ...weitereSkills] = skills;
console.log(erste);           // "HTML"
console.log(weitereSkills); // ["JavaScript", "React"]
```

Merke: Das Rest-Element muss beim Array-Destructuring an letzter Stelle stehen.

Rest bei Objekten

Ausgewählte Properties werden herausgenommen; die übrigen landen in einem neuen Objekt.

```
const developer = {
  name: "Marcel",
  role: "Frontend-Entwickler",
  aktiv: true
};

const { name, ...developerData } = developer;
console.log(name);           // "Marcel"
console.log(developerData); // { role: ..., aktiv: true }
```

Rest in Funktionsparametern

Eine Funktion kann damit beliebig viele Argumente als Array empfangen.

```
function zeigeSkills(ueberschrift, ...skills) {
  console.log(ueberschrift);
  console.log(skills);
}

zeigeSkills("Frontend", "HTML", "CSS", "JavaScript");
// skills: ["HTML", "CSS", "JavaScript"]
```

Merke: Ein Rest-Parameter muss der letzte Parameter der Funktion sein.

4. Spread oder Rest?

Frage	Spread	Rest
Was passiert?	Werte werden verteilt.	Werte werden gesammelt.
Typischer Kontext	Neues Array/Objekt, Funktionsaufruf	Destructuring, Parameter
Denkhilfe	„Nimm alles daraus.“	„Pack den Rest hier hinein.“
Syntax	<code>[...array] / {...objekt}</code>	<code>[a, ...rest] / {a, ...rest}</code>

5. Abschlussübung

Ziel: Skills erweitern, das Developer-Objekt ergänzen und anschließend name vom Rest der Daten trennen.

```
const frontendSkills = ["HTML", "CSS", "JavaScript"];

const developer = {
  name: "Marcel",
  role: "Frontend-Entwickler"
};

const allSkills = [...frontendSkills, "React", "Git"];

const updatedDeveloper = {
  ...developer,
  skills: allSkills
};

const { name, ...developerData } = updatedDeveloper;

console.log(name);           // "Marcel"
console.log(developerData); // { role: ..., skills: [...] }
```

Was passiert hier? `...frontendSkills` ist Spread, `...developer` ist ebenfalls Spread und `...developerData` ist Rest. Entscheidend ist also nicht das Zeichen, sondern der Kontext.

Kurz-Check

- Spread kopiert bzw. verteilt Werte.
- Rest sammelt übrige Werte.
- Bei doppelten Objekt-Properties gewinnt der letzte Wert.
- Rest steht immer am Ende.